

Contents

Security architecture — the network logging device and process	1
1. Objective and trust boundary	1
2. Physical layout and device	2
3. Timing isolation and metadata removal	3
4. Requests, responses, and packets	4
4.1 Packet structure	4
4.2 Packets are self-locating	4
4.3 No undercounting: per-packet information sums correctly	4
4.4 Why subsets can be queried	5
5. Certificate generation	5
6. Integration with the broader prover system	6
7. Fault tolerance	7
8. Information flow through the device	9
9. Security requirements	10
Certificate requirements (what the verifier assumes)	10
Device requirements (what upholds them)	11
10. Threat table	12
11. Residual risks and open items	14

Security architecture — the network logging device and process

This document is the security architecture for the **interlock**: the network logging device that sits on a compute node's only network path, and the surrounding process (prover logging, verifier challenges) that turns its output into verified statements about the node's traffic.

It complements the existing docs rather than replacing them:

- `verification-protocol.md` — the protocol itself (objects, challenge flow, recomputation options).
- `prod_ilock_core.md`, `prod_canon_proc.md` — the production core's block designs (further block detail lives in the RTL headers under `gateway/src/src_hdl/`).
- `ethernet-frame-sanitization.md` — the per-field 802.3 sanitization rules.
- `recomp_ilock_core.md`, `recomp_feed.md` — the recomputation interlock.
- `gateway-crypto-backend.md` — key custody and side-channel decisions.

Implementation references are to the **production core on main** (`gateway/src/src_hdl/fabric_bridge` and its blocks). An earlier conformance core (`gateway/src/core/` on `feature/gateway-core`) validated the certificate pipeline byte-for-byte against the Python golden model; it is superseded by the production line but remains the cross-checked reference for the hash/certificate math.

Terminology note: this document uses **epoch** for the certificate window (N buckets; one certificate per epoch — $1\text{ s} / 1000 \times 1\text{ ms}$ buckets in the production parameters: `TIMER_END = 79_999` at the 80 MHz fabric clock, `BKTS_PER_CERT = 1000`). The other docs call this the “cert window” or “second”.

1. Objective and trust boundary

A compute node (the **prover's** hardware) is *quarantined*: every byte it exchanges with the outside world crosses the interlock, and nothing else about the node is observable from outside. The interlock makes the node's communication **auditable after the fact** without trusting

the prover: it emits **traffic certificates** — one per epoch — each attesting to exactly which packets traversed the device during that epoch, represented by per-packet commitments, and signed by an on-device secret key shared only with the **verifier**.

Party	Trust status	Holds
Interlock	Trusted for enforcement; assumed correct logic, tamper-proof state, secret key	HMAC key k , bucket timer, running hash state + one bucket buffer per direction. Stores no history.
Prover (frontend + compute)	Untrusted; physically hosts the interlock	Full traffic logs, payload keys, weights
Verifier	Trusted by itself; remote	Shared key k , anchor log of nonce→certificate round trips

The prover’s physical access to the device is in scope: the threat model includes power/EM side channels against the key and attempts to reset or rewind device state. The verifier’s end goal (property P3 of verification-protocol.md) is a sound upper bound U on unexplained information leaving the node; everything below exists so that per-packet accounting of U is *complete* — no information leaves the quarantine outside the committed packets.

2. Physical layout and device

The interlock is a Microchip **PolarFire MPF300TS** FPGA (S-grade; MPF300-EVAL-KIT for the prototype) wired in-line on the compute node’s only network path, with one Ethernet port per side:

```

verifier ↔ prover frontend ↔ [ port 0 | PolarFire fabric | port 1 ] ↔ compute node
                                interlock device                    (quarantined)

```

- **Two 1G Ethernet ports** (VSC8575 PHYs over SGMII via IOD CDR), one facing the prover frontend, one facing the quarantined compute node. The device is the only Layer-2 endpoint either side sees; there is no other cable, radio, or shared medium into the quarantine.
- **All-fabric datapath.** Forwarding, hashing, buffering, and certification are plain SystemVerilog in FPGA fabric at an 80 MHz fabric clock — no CPU, no firmware, no software stack on the trusted path (the Mi-V soft CPU in the build serves only off-datapath PHY bring-up). The TCB is a small set of inspectable RTL blocks (§8).
- **On-die key security: PUF + encrypted flash.** PolarFire has no battery-backed key RAM; its security primitives are an **SRAM-PUF** (a device-unique silicon fingerprint from which key-wrapping keys are derived — never stored, regenerated at each power-up) and **sNVM** (56 KB secure flash whose pages can be PUF-encrypted and authenticated). The HMAC key at rest exists only as PUF-wrapped ciphertext in sNVM; the plaintext key exists only while powered. The same sNVM holds the persisted epoch counter that makes restarts non-rewinding (§7). On-chip **tamper detectors** (voltage, temperature, clock/JTAG monitors) can trigger **zeroization**, destroying the sNVM contents — key death on active tamper, without a battery.
- **On-die crypto hardware** (Athena TeraFire cryptoprocessor, present on S-grade parts) is the phase-2 home of the HMAC: validated DPA/SPA countermeasures against the physically-present adversary (gateway-crypto-backend.md).
- The prover hosts the device in its facility. Physical substitution (unplugging the interlock, routing around it) is countered by the process, not the box: certificates must tile the timeline under a key the prover never learns, so traffic that bypasses the device is traffic with no certificate — a violation, not an escape (§9, C1/C3).

For the recomputation option, a second interlock of the same construction (`recomp_ilock_core`, a structural sibling of the production core) mediates the verifier-enclosure boundary at challenge time (§6).

3. Timing isolation and metadata removal

Raw Ethernet between two hosts carries far more information than the payload bytes: addresses, header quirks, padding content, inter-frame spacing, microsecond-level send times. The interlock’s first job is to destroy every such channel, leaving only what the protocol explicitly commits.

Metadata removal — sanitize, terminate, re-originate. Each port is a full L2 endpoint, and every 802.3 field is forced to a known-good canonical form (`ethernet-frame-sanitization.md`): DST and SRC are forced/whitelisted to exactly one address each; the LENGTH field is enforced against the bytes actually received (mismatch = drop); PAD is dropped where unnecessary and overwritten with a fixed value where the 64-octet minimum requires it; the FCS is discarded and recomputed over the sanitized frame. Only the DATA field — the canonical packet (§4) — crosses, verbatim. One residual is explicitly accepted rather than eliminated: frame segmentation (the LENGTH value itself) can carry on the order of a bit per frame at a cost of ~4 unexplained bytes; it is undetectable at frame level and is charged in the unexplained-information accounting instead (§11).

Timing isolation — quantize egress to the bucket. Time is divided into 1 ms buckets by a free-running on-device timer. Egress in each direction is staged through a **ping-pong bucket buffer** (`batch_buffer.sv`): packets accepted during bucket *b* accumulate in one bank while the other bank — bucket *b*–1’s contents — drains to the wire; the banks swap exactly on the tick. An admission guard accepts a packet’s first beat only if enough cycles remain before the tick (else it stalls until after the swap), so no record straddles banks; a drain still running at the next tick is preempted — the cut is closed and the un-emitted remainder dropped (§7). The outside world therefore observes only **bank contents at bucket changeover points**, never sub-bucket timing. The timing channel that remains (which bucket a packet lands in) is explicit, committed (the in-packet bucket field, §4), and capacity-bounded by the bucket width and the bank size.

No side observables. Invalid packets are silently suppressed (header-check failures never leave `canon_proc`; payload-check failures are gated out by `axis_pkt_gate`) — no drop counters, no status frames. (A dropped packet never exits the node, so it needs no accounting; the prover detects its own drops by comparing its logs on the two sides.) The interlock’s own emissions are limited to: sanitized forwarded frames, certificates, and **sync packets**. Sync packets (one per tick, reserved ID = 1) are each direction’s time reference, and each is routed back toward **that direction’s own sender**: the frontend’s sync carries the arrival phase (FIRST_ARR) of the frontend’s *own* traffic, and the compute node’s sync flows inward — so neither stream can carry compute-influenced information outward. The certificate is exactly prover-predictable field-for-field, so the device’s total covert-output surface reduces to nonce-latch timing (≤ 1 bit per nonce update) plus the HMAC tag bits (auditable by slot-bound cut-and-choose, see `verification-protocol.md`).

The net effect, stated as the isolation invariant the rest of the document builds on:

No information leaves the quarantine other than the contents of the ping-pong buffer at the bucket changeover points — plus bucket-granular timing, which is committed and capacity-bounded.

4. Requests, responses, and packets

4.1 Packet structure

All traffic is structured as canonical packets with a fixed 64-byte cleartext header and an end-to-end-encrypted payload the interlock never decrypts. A canonical packet must fit a single Ethernet DATA field — no segmentation (that is what neutralizes segmentation as a free channel). Fields (`canon_pkg.sv`):

- **Request (in):** `pld_len || bucket || id || reference || key_commit || payload` — `key_commit = H(key material)` pins, at request time, the keys any later recomputation may use; `reference` points to an earlier request whose context this one extends (multi-turn, long inputs/outputs — chain logic lives entirely outside the device, which hashes the field like any other bytes).
- **Response (out):** `pld_len || bucket || id || reserved || payload` — a response binds to its request by `id` and inherits its context through it.

The `id`'s LSB flags inference vs. other traffic; `ids` 0 and 1 are reserved for control: an inbound `id = 0` packet carries a verifier **nonce** (latched, never forwarded), `id = 1` marks the device's own sync packets, and a zero header flags a certificate frame.

Validity rules (enforced by `canon_proc` as the packet streams through; violators are suppressed before the buffer, or gated at `tlast` for payload-length mismatch): payload length within range and, for inference packets, a whole number of token units; `ids` strictly increasing per direction (the response comparator resets per bucket, since responses finish out of order); `reference` earlier than `id`; `reserved` fields zero; and the declared **bucket exact-matches** the device's current bucket.

4.2 Packets are self-locating

The validity rules are chosen so that two in-packet fields pin each accepted packet's position in the timeline:

- **Bucket** — accepted only on exact match with the device counter (the check keys directly off the tick, independent of downstream backpressure), so an accepted packet's recorded bucket is its true-time bucket.
- **Id** — strictly monotonic per direction within a bucket, so within a bucket there are no duplicate `ids` per direction and sorting by `id` reconstructs the exact order the device folded them. (The same response `id` may recur in *different* buckets; the bucket field disambiguates, so (direction, bucket, id) is globally unique.)

Consequence: the **unordered set** of accepted packet data suffices to reconstruct the ping-pong buffer contents at every changeover point of the epoch — group by bucket, sort by `id`, and the per-bucket sequences (hence every digest in §5) are reproduced byte-exactly.

4.3 No undercounting: per-packet information sums correctly

Reconstruction from *unordered* data means all information in the buffer snapshots is contained in the packets themselves — there is no residual degree of freedom that carries information per-packet accounting would miss:

- If within-bucket **ordering** were attacker-chosen and observable, a prover could encode extra bits in the permutation ($\log_2 n!$ bits per bucket of n packets) invisibly to per-packet accounting. Sort-by-`id` removes this freedom: any reconstructed order is a pure function of packet contents.
- If **bucket placement** were free, cross-bucket arrangement would be a channel. Exact-match declaration removes it: placement equals true arrival time, already accounted as the (bounded, committed) timing channel of §3.

So the total information crossing the boundary is exactly the **sum over packets of their individual information content** — the quantity the challenge protocol samples.

4.4 Why subsets can be queried

The structure above is what lets the verifier audit a *sample* instead of the whole log, and still get an unbiased estimate:

- **Openable in small slices.** Commitments are hierarchical (record $\rightarrow$ bucket digest $\rightarrow$ epoch digest, §5), so one challenged byte opens with: one certificate, that epoch’s bucket-digest list, one bucket’s records, and one packet’s header + $H(\text{payload})$ — a few hundred KB, no payloads.
- **Unbiased sampling.** The verifier draws uniform (bucket y , byte x) over $[B - \text{window}, B] \times [0, C)$. Uniform-over-capacity is size-weighted sampling: every transmitted byte is equally likely to be challenged, so the sampled per-byte unexplained-information estimate is unbiased over all traffic; an x beyond the bucket’s actual content is a cheap liveness check. Because packets are self-locating (§4.2), the prover cannot steer which packet “answers” a given (y, x) — cumulative lengths within the reconstructed bucket order determine it.
- **No undercount under sampling.** By §4.3 the total information is the sum of per-packet contributions, so a uniform per-byte sample scales to a sound whole-traffic bound; there is no mass hidden in ordering, placement, or unlogged observables for sampling to miss.

5. Certificate generation

Every epoch the device emits one certificate committing both directions, computed incrementally as packets stream through (traffic_commit + cert_build; nothing stored beyond the running state):

1. per packet: record = length || SHA256(header || $H(\text{payload})$)
2. per bucket: bucket_hash – running SHA-256 over the bucket’s records
(closed by the in-band boundary marker)
3. per epoch: overall – running SHA-256 over the N bucket hashes,
one per direction (overall_req, overall_rsp)

```
m = version || interlock_id || bucket_start || num_buckets
    || overall_req || overall_rsp || nonce || prev_τ
τ = HMAC_k(m)
certificate frame = [ zero canonical header (id = 0) ] || m || τ
```

Three binding layers: the **packet commitment** binds the packet data (header — including the declared bucket, id, reference, and key commitment — and payload, via $H(\text{header} || H(\text{payload}))$); the **certificate** binds the epoch’s packet commitments and attests their integrity; the **on-device key** makes the attestation unforgeable. $H(\text{payload})$ is kept a separable leaf so a challenge can reveal a packet’s structure without its payload, and so it doubles as the recomputation certificate’s public input (P5, P3).

Certificates are chained. Each signed body includes the previous certificate’s tag (prev_τ), so the certificate stream is a hash chain: any anchor the verifier holds transitively fixes the entire prior sequence, and no certificate can be dropped, reordered, or substituted between two anchors without breaking the chain. Together with bucket_start || num_buckets tiling the bucket timeline, this gives one append-only history per key (C2, C3).

Commit placement is direction-asymmetric, deliberately (see the §8 figure): requests are committed **before** the bucket buffer — the certificate binds what was *accepted into* the quarantine — while responses are committed **after** it, on the drain side — the certificate

binds exactly the bytes that *left* the quarantine. A response drain preempted at the tick (§7) therefore never commits its unsent remainder: the attested outbound log and the wire are identical by construction, which is the direction where it matters.

The nonce latch needs no authentication. The verifier’s nonce arrives as an inbound `id = 0` control packet; `canon_proc` latches its key-commit field (low 128 bits) and suppresses the packet — it is never forwarded, and the capture is gated on a non-fractional header so a spliced frame can never latch one. The device never verifies a MAC: the verifier credits only nonces it generated, so injected or replayed nonces are indistinguishable from dropped ones — both read as staleness, the failure the mechanism exists to expose. The interlock generates MACs but never checks one, keeping it off the HMAC-oracle surface.

Every hash is recomputed by the device from wire bytes; no prover-supplied value enters a certificate. Conversely, every certificate field is deterministic given the accepted stream and the nonce — the prover can (and does) byte-audit each certificate as it arrives. Certificates are emitted continuously, unprompted — emission is never a response to a query, so query timing cannot become a device output.

6. Integration with the broader prover system

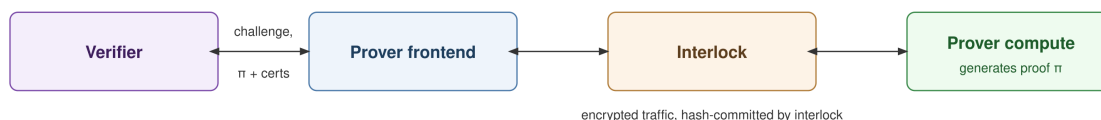


Figure 1: Workload path: verifier — prover frontend — interlock — prover compute. Traffic is end-to-end encrypted between frontend and compute; the interlock hash-commits it in transit. At challenge time the ZKP option has prover compute generate the proof π , forwarded with the certificates.

Division of labor around the device:

- **Interlock:** on-the-fly enforcement and certification only. Holds one bucket buffer per direction plus running hash state, stores no history, answers no queries.
- **Prover frontend:** the untrusted convenience replica. It logs both directions’ packets, key material, and every certificate for the challenge window (~30 days); tracks the device’s bucket clock from the per-tick **sync packets** (the `FIRST_ARR` field tells it how deep into a bucket its probe actually landed — feedback it iterates on to widen its send window; at most one sync in flight), so it can declare buckets that exact-match; byte-audits each certificate on arrival; and serves challenge openings.
- **Verifier:** keeps an **anchor log** of nonce→certificate round trips (with receive times on its own clock), verifies the certificate chain and bucket tiling, checks counter rate against wall clock, samples challenges (§4.4), recomputes every hash over the opened slices, and consumes the recomputation certificate for the sampled pair.

The authority asymmetry (verification-protocol.md, *Two invariants*): the certificate is the sole authority; the prover’s log is best-effort. A challenge the log cannot satisfy consistently with the certificate is a failed challenge charged as unexplained information — prover logging failures degrade the prover’s standing, never soundness (§7).

At challenge time, the opened traffic values (H1_in, H2, H1_out) bridge into a **recomputation certificate** attesting the response carries at most U unexplained bits:

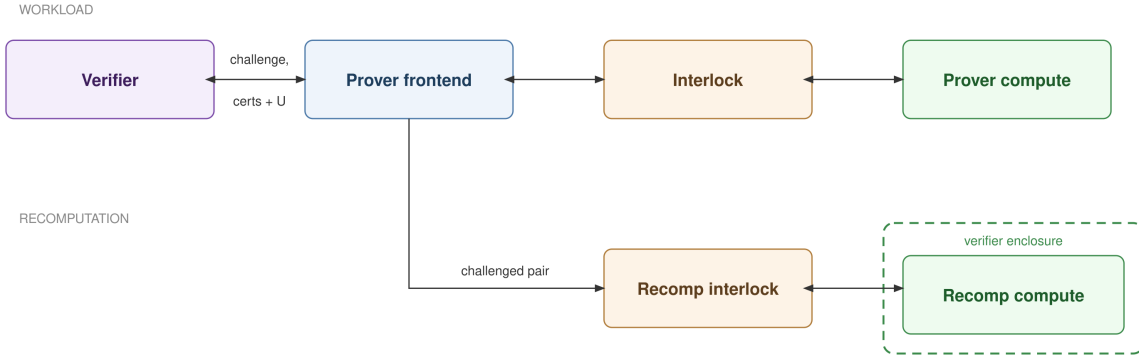


Figure 2: Prover-recomputation option: the same workload track, plus a challenge-time track — the frontend stages the challenged pair through a second recomputation interlock into the verifier enclosure.

The recomputation interlock (`recomp_ilock_core`) is a structural sibling of the production core with one challenge direction: the frontend stages the challenge slice through the same `de-frame` → `canon_proc` → `commit` → `bucket-buffer` path, then **recomp_feed** forwards the context into the enclosure and runs the challenged response as an **estimate/reveal loop** — the enclosure commits a probability estimate for each position *before* that position’s token is revealed; the device checks each estimate is a valid sub-distribution (a malformed or missing estimate is charged at the value space’s maximum surprisal, so cheating never under-charges), accumulates the running total $\hat{U}$ over length, timing, and every token, and feeds $\hat{U}$ plus its own ingress digest to `cert_build`.

It needs no traffic *timeline* — no epochs, no persisted counter; its certificate binds to the challenge nonce. But it is **not stateless**: its certificate must attest to *exactly what information entered the recompute node prior to each step*, which is inherently session state — the ingress traffic digest since enclosure seal/wipe, the commit-before-reveal ordering, and the $\hat{U}$ accumulator. Without that state, a covert payload staged into the enclosure before or between predictions could be laundered as “explained.” Reset of the recomputation device mid-session must abort the session, never resume it. (The recomputation certificate format is still a placeholder in RTL — see §11.)

7. Fault tolerance

The design principle is **fail closed toward soundness, degrade gracefully toward availability**: no failure mode may yield a false certificate or an ambiguous timeline; failures instead cost traffic, prover standing, or (in the limit) the device’s ability to certify at all.

- **Restarts never rewind the timeline.** The key survives power cycles (PUF-wrapped in sNVM, §2), so uniqueness cannot rest on state evaporating; it rests on a **persisted monotonic epoch counter** with never-rewind boot logic. The counter is written to sNVM every W epochs; on boot the device resumes at `persisted + W` (a safety margin past any epoch it could have certified), so no epoch is ever certified twice (C2) even across arbitrary power cycles. The restart itself is unhideable: the skipped epochs are a gap in the certificate chain and bucket tiling, which the verifier treats as unattested time — worst-case information charge for the gap, or grounds for re-inspection (policy, not

mechanism). Rolling the persisted counter back would require rewriting authenticated sNVM inside the device’s security boundary — the same trust as key storage itself. Active tamper (detector-triggered) escalates to zeroization: key death, ending the stream permanently. (*This key/counter-persistence layer is design; the RTL currently takes the key on a port — §11.*)

- **Bucket boundaries never wait.** The tick is free-running; `canon_proc` moves to the new bucket the instant it fires, independent of downstream backpressure, and signals the boundary in-band as an unambiguous empty beat. Congestion can delay *packets*, never *time*.
- **Buffer admission and preemption.** The admission guard stalls a packet at its first beat rather than letting a record straddle banks; if the guard is ever violated, the in-flight record is abandoned rather than half-committed. A drain still running at the next tick is preempted: a termination beat closes the cut stream and the remainder is dropped — and because responses are committed on the drain side (§5), a preempted remainder is neither released nor attested. Loss is always visible to the endpoints, never absorbed silently into commitments.
- **Honest drops are avoidable and non-fatal.** Exact-match buckets drop boundary-straddling declarations; the honest frontend avoids this by calibrating against the sync packets’ `FIRST_ARR` feedback and not sending within a guard window of the tick. A drop that happens anyway loses one packet’s traffic; it never corrupts a certificate. Recovery is the endpoints’ job via a new request.
- **Sync-packet loss is harmless.** At most one is in flight; the frontend’s calibration loop free-runs between syncs and re-anchors on the next one. A mis-synced sender only mis-declares and drops itself — visible to it, irrelevant to soundness.
- **Prover storage loss degrades to U.** A missing or corrupt log slice, certificate copy, or reference-chain link makes the corresponding challenge unanswerable-consistently — charged as unexplained information. No special repair path exists or is needed.
- **Recomputation-side faults never under-charge.** A malformed estimate, a failed normalization check, or a truncated estimate frame charges the position at the minimum representable probability (at least the value space’s max entropy); the loop continues — no fault, no abort, no discount.

8. Information flow through the device

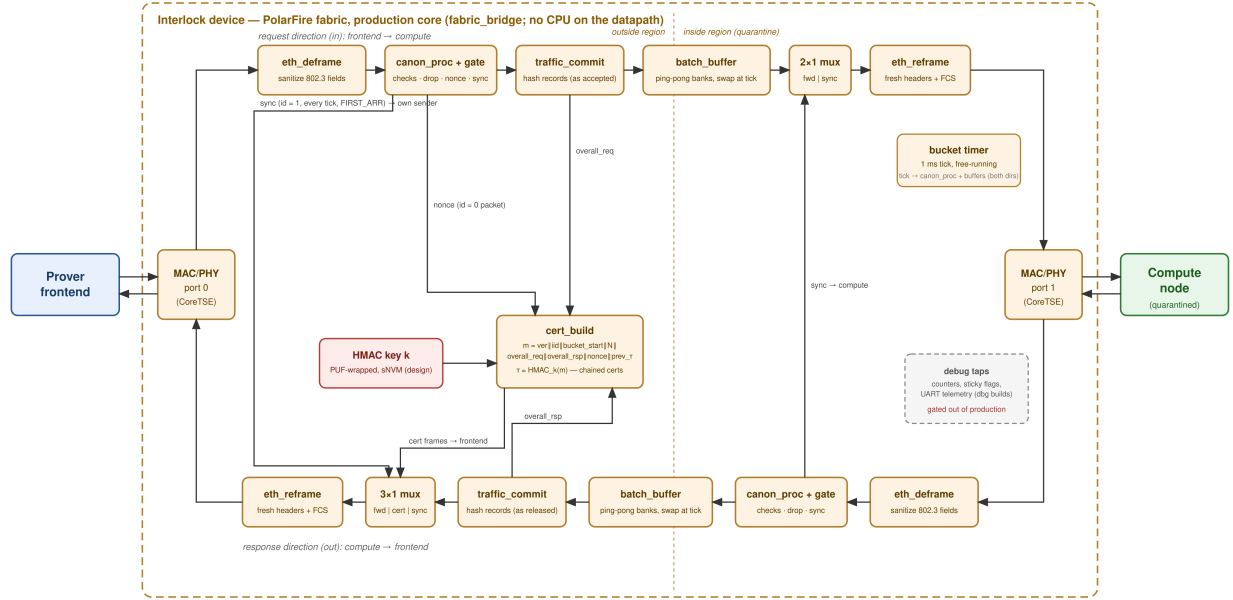


Figure 3: Information flow through the production interlock core (fabric_bridge). Both directions are sanitized, checked, and staged through bucket-quantized ping-pong buffers straddling the quarantine boundary; requests are hash-committed as accepted (before the buffer), responses as released (after it). Both digests, the latched nonce, and the previous tag feed cert_build, whose chained certificate frames join the frontend-bound mux; per-tick sync packets flow back to each direction's own sender.

Components, in datapath order (all gateware/src/src_hdl/ on main):

Block	Role	Security relevance
MAC/PHY (CoreTSE ×2)	L1/L2 per port; FCS check	Vendor IP at the edge; everything it passes is re-validated in fabric
eth_deframe / eth_reframe	802.3 sanitization: force DST/SRC, enforce LENGTH, strip PAD; re-originate with fresh headers + recomputed FCS	Metadata removal (§3): only canonical DATA crosses
canon_proc (+ axis_pkt_gate)	Header shift-register: length, id, reference, reserved, bucket checks; in-band boundary markers, nonce capture (id = 0), sync emission (id = 1); gate suppresses payload-check failures	Enforces §4.1 — the premises of the reconstruction argument; drops are silent

Block	Role	Security relevance
traffic_commit (record_layer, serializer, sha256_msg)	Per-packet record → running bucket digest → epoch digest; rate-matched, never back-pressures the datapath	The commitment hierarchy (§5); requests as accepted, responses as released
batch_buffer	Ping-pong banks; swap exactly at tick; admission guard; drain preemption	Timing isolation (§3); outside sees only changeover snapshots
cert_build (+ crypto/: sha256_core, sha256_msg, hmac_sha256)	Assemble $m, \tau = \text{HMAC}_k(m)$, emit the 0-id certificate frame; prev τ chains certificates	The attestation (§5); the only secret-bearing operation
axis_mux3 / 2×1 mux	Arbitrate forwarded / cert / sync per egress	Fixed, prover-predictable egress set
Bucket timer (in fabric_bridge)	Free-running 1 ms tick to canon_proc + buffers	Authoritative time base; no external set path
Key + counter persistence	PUF-wrapped key in sNVM; persisted monotonic epoch counter; tamper → zeroization	Reset resistance (§7); key custody (§2) — design; RTL takes the key on a port
recomp_ilock_core / recomp_feed / norm_chk	Challenge-time sibling: ingress commit + buffer + estimate/reveal loop + $\hat{0}$	The recomputation interlock (§6); session-stateful by design
Debug taps (pkt_counter, sticky_bit; UART telemetry on debug branches)	Bring-up observability	Must be gated out of production (§11)

Assurance: the production blocks carry their own testbenches (timeline-compressed via the timer parameters); the certificate math is additionally anchored by the earlier conformance core, which was verified byte-for-byte against the Python golden model (prototype/interlock.py) under cocotb and independently audited (A-G5). The final gate for any image is the certificate-byte comparison against the golden model on silicon.

9. Security requirements

Certificate requirements (what the verifier assumes)

#	Requirement	Statement
C1	Completeness	The certificate for epoch e commits <i>every</i> packet that traversed the interlock during e , both directions. No byte crosses uncommitted.
C2	Uniqueness (reset resistance)	Over the lifetime of key k , at most one certificate is ever generated for a given epoch. No reset, power cycle, or state rewind may allow an already-certified epoch to be certified again.

#	Requirement	Statement
C3	Continuity (monotonic order)	Certificates are generated only in strictly increasing epoch order and form an unbroken chain (<code>prev_τ</code>) whose <code>bucket_start</code> values tile the bucket timeline — no gap or overlap the prover can hide.
C4	Freshness	A certificate echoing verifier nonce <code>r</code> was generated after <code>r</code> was issued.
C5	Bounded worst-case bandwidth	Per epoch and direction at most $N \times$ (bank capacity) committed bytes, and the epoch counter advances at most $\sim 2 \times$ wall-clock rate, so elapsed real time caps total volume between checks. Slowdown needs no assumption — it is directly visible in the anchor log.
C6	Queryability	The verifier can audit recent epochs on demand: send a nonce, receive the next certificate echoing it and stamping the current epoch, then challenge any epoch back to the last anchor.
C7	Unforgeability	Only the interlock (and the verifier itself) can produce a valid tag; a certificate accepted under <code>k</code> and <code>interlock_id</code> originated from that device.

C2 + C3 give the verifier a single, append-only timeline per key: exactly one history, fixed before any challenge is known (P2). C4 + C5 anchor it to wall clock (P4): history cannot be stretched, replayed, or run ahead, and bursts between checks are bounded. The query pattern exercising C4–C6: every few days, anchor with a fresh nonce, verify the chain, tiling, and rate, then challenge a uniformly random epoch from the unaudited interval — the prover cannot predict which epoch must hold up.

Device requirements (what upholds them)

#	Device requirement	Mechanism	Upholds
D1	Correct enforcement logic	<code>canon_proc</code> checks + gate; commitment pipeline; tested and audited (§8)	C1, C3, C5 (caps), §4’s premises

#	Device requirement	Mechanism	Upholds
D2	Information isolation	802.3 sanitization, bucket-quantized egress, silent drops, sender-directed sync (§3)	C1's converse — nothing escapes commitment
D3	Binding	record → bucket → epoch → HMAC, all recomputed from wire bytes; prev_τ chaining (§5)	C1, C3, C7
D4	Tamper-proof, non-rewindable state	Persisted monotonic counter in authenticated sNVM, skip-ahead boot; free-running timer with no set path while powered; tamper → zeroization (§7)	C2, C3, C5 (rate)
D5	Key protection	PUF-wrapped key at rest in sNVM; phase 2: TeraFire DPA-resistant HMAC (§2)	C7 — and transitively everything

Summarized: **correct logic, state that cannot be tampered with or rewound, and a key that cannot be read.**

10. Threat table

Attack	Defense	Requirement
Exfiltrate outside packet framing (bypass the wire)	Physical quarantine: interlock is the only network path	scope (§2)
Exfiltrate via 802.3 header fields, padding, or malformed frames	Per-field sanitization: forced DST/SRC, enforced LENGTH, neutralized PAD, recomputed FCS	C1 (D2)
Exfiltrate via frame segmentation (LENGTH choice)	Accepted residual, charged in unexplained-information accounting (~4 unexplained bytes per covert bit)	P3
Exfiltrate via packet ordering within a bucket	Strictly increasing ids; order is a pure function of content (§4.3)	C1
Exfiltrate via bucket placement / clock games	Exact-match declared bucket; placement = true time; timing capacity fixed at bucket granularity (§3)	C1, C5

Attack	Defense	Requirement
Exfiltrate via sub-bucket timing	Ping-pong bucket buffers release only at changeover (implemented on main; silicon verification pending)	C1 (D2)
Exfiltrate via drop patterns or device telemetry	Silent suppression; no drop observables on the production path; debug taps gated out of production	C1 (D2)
Rewrite history after seeing a challenge	Certificate fixed within one epoch of traffic; challenges reference committed epochs only	P2 (C1+C3)
Reset the device, replay or fork the timeline	Persisted monotonic counter + skip-ahead boot: restarts skip epochs, never repeat them; counter rollback requires defeating sNVM authentication; active tamper → zeroization	C2
Skip, reorder, or substitute certificates	prev_τ hash chain + bucket_start tiling + verifier anchor log	C3
Serve stale certificates	Nonce echo; verifier credits only its own nonces	C4
Run the clock fast to pre-compute history / buffer bursts	Counter-rate check vs verifier wall clock (~2× bound); per-bucket bank capacity	C5
Forge or tamper certificates	HMAC-SHA-256 over the full body under k	C7
Extract k via power/EM side channels	Phase 2: DPA-resistant User Crypto + sNVM; streaming path carries no secret	C7 (D5)
Fabricate an “input” that contains the covert output	Input binding at challenge time: context chain opens against certificates; single-use responses	P3 (process)
Launder the covert output through recomputation	Session-stateful recomputation interlock: ingress digest since seal, H2-gated key entry, commit-before-reveal, sub-distribution check with max-entropy charge on any malformation	P3 (\$6)

Attack	Defense	Requirement
Use the interlock itself as a covert transmitter	Every cert field prover-predictable; sync packets flow only to each direction's own sender; nonce-latch timing ≤ 1 bit/update; tag release auditable by cut-and-choose	P5 (D2)

11. Residual risks and open items

1. **Silicon verification pending.** The production core (sanitization, canon_proc, batch_buffer timing isolation, chained certs) exists on main and in testbenches; the currently-flashed image predates it. The isolation and certificate claims hold for hardware only after the on-silicon certificate-byte and timing-release checks pass.
2. **Debug observability in production.** Bring-up needs counters, sticky flags, and (on debug branches) UART telemetry — exactly what D2 forbids in deployment. Required: a build-level gate that provably excludes all debug taps from a production bitstream.
3. **Phase-1 key custody and counter persistence.** cert_build takes the key on a port; the PUF/sNVM wrapping, the persisted-counter FSM, and the tamper/zeroization hookup are design, not yet built. All current results read as “assuming key secrecy,” which phase-1 hardware does not yet earn against a physical adversary. **sNVM write endurance** constrains the persistence cadence W : secure-flash pages have limited program cycles, so W and page wear-leveling must be engineered against the datasheet limit (larger W costs a larger skipped gap per restart, not soundness). *Note — doc drift:* verification-protocol.md still describes a battery-backed domain (assumption 1) and calls the recomputation interlock “stateless”; both should be corrected per §7 and §6. The feature/gateway-core branch docs (bucket-declaration-spec.md, gateway-debug.md) describe the superseded conformance line.
4. **Working-state tamper-proofness.** The powered running state is assumed untouchable (D4). The persisted counter covers *restarts*; active probing or glitching of live state relies on the tamper detectors, whose coverage against fault injection is deferred to the physical-hardening pass, scoped with D5.
5. **Bucket-counter width.** The canonical bucket field is 32-bit (canon_pkg.sv); at 1 ms ticks it wraps after ~ 49.7 days. The prev_τ chain keeps the *order* unambiguous across a wrap, but bucket-numbered challenge addressing and the C2 “one certificate per epoch number” statement need either a wider counter or an explicit wrap/era policy for key lifetimes beyond ~ 7 weeks.
6. **Honest drop rate.** Believed negligible with sync-packet calibration and a sender-side guard window; to be measured on silicon.
7. **Speedup bound enforcement.** C5 assumes the device oscillator cannot run more than $\sim 2\times$ fast; the enforcement (oscillator choice, on-die clock monitoring, or tighter anchor cadence) is not yet specified.
8. **Recomputation certificate format is a placeholder.** recomp_ilock_core currently emits the production traffic-cert layout with 0 standing in for the response digest; the pinned format (nonce, H1_in, H2, H1_out, n_units, U) needs its own builder before recomputation results are protocol-valid.